ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



HỆ THỐNG NHÚNG (CO3054)

BÁO CÁO BÀI THỰC HÀNH 1

LÀM VIỆC VỚI LED & CƠ CHẾ ĐA NHIỆM

Lớp: L02 - Học Kỳ HK261

Ngày nộp: 27/10/2026

GVHD: Thầy Lê Phương Nam

Sinh Viên	MSSV
Phạm Công Võ	2313946

TP.Hồ Chí Minh, Tháng 10/2026

THÔNG TIN THỰC HIỆN VÀ ĐÁNH GIÁ KẾT QUẢ

BÀI THỰC HÀNH 1

STT	Họ và tên	MSSV	Nhiệm vụ	Kết quả	Chữ ký
1	Phạm Công Võ	2313946	• Hiện thực toàn bộ mã nguồn Bài 1 đến Bài 5 • Thực nghiệm trên Kit BKIT ARM4 • Soạn thảo tài liệu báo cáo	100%	

Mục lục

1 Tổng Quan và Mục Tiêu Bài Thực Hành 1 **3**
1.1 Giới thiệu . . . 3
1.2 Mục tiêu . . . 3
1.3 Ánh xạ phần cứng trên Kit BKIT ARM4 . . . 3

2 Nội Dung Chi Tiết Các Bài Tập Thực Hành **4**
2.1 Bài 1: Nháy LED với thời gian sáng / tắt khác nhau . . . 4
2.1.1 Nguyên lý giải quyết và thiết kế luồng . . . 4
2.1.2 Mã nguồn chương trình (`src/main.c`) . . . 4
2.1.3 Kết quả thực nghiệm trên Kit và Log Serial Monitor . . . 5
2.1.4 Nhận xét và đánh giá . . . 5
2.2 Bài 2: Đồng bộ LED3 và Output Y0 qua biến dùng chung . . . 6
2.2.1 Nguyên lý giải quyết và cơ chế đồng bộ biến chung . . . 6
2.2.2 Mã nguồn chương trình (`src/main.c`) . . . 6
2.2.3 Kết quả thực nghiệm trên Kit và Log Serial Monitor . . . 8
2.2.4 Nhận xét và đánh giá . . . 8
2.3 Bài 3: Các mẫu nháy độc lập (Independent Blink Patterns) . . . 9
2.3.1 Nguyên lý giải quyết và cơ chế đa nhiệm ưu tiên . . . 9
2.3.2 Mã nguồn chương trình (`src/main.c`) . . . 9
2.3.3 Kết quả thực nghiệm trên Kit và Log Serial Monitor . . . 11
2.3.4 Nhận xét và đánh giá . . . 11
2.4 Bài 4: Một hàm thread — Nhiều đèn (Truyền tham số cho thread) . . . 12
2.4.1 Nguyên lý thiết kế Generic Thread và ép kiểu con trỏ . . . 12
2.4.2 Mã nguồn chương trình (`src/main.c`) . . . 12
2.4.3 Kết quả thực nghiệm trên Kit và Log Serial Monitor . . . 14
2.4.4 Trả lời câu hỏi lý thuyết của đề bài . . . 14
2.5 Bài 5: Mô phỏng hệ thống đèn giao thông (Traffic Light Simulation) . . . 15
2.5.1 Nguyên lý thiết kế máy trạng thái và mảng cấu hình Table-driven . . . 15
2.5.2 Mã nguồn chương trình (`src/main.c`) . . . 15
2.5.3 Kết quả thực nghiệm trên Kit và Log Serial Monitor . . . 17
2.5.4 Nhận xét và đánh giá . . . 18

Tài liệu tham khảo **19**

1 Tổng Quan và Mục Tiêu Bài Thực Hành 1

1.1 Giới thiệu

Bài thực hành 1 làm quen với hệ điều hành thời gian thực Zephyr RTOS trên Kit thí nghiệm BKIT ARM4 (STM32F407ZGT6, ARM Cortex-M4 168 MHz). Thay vì sử dụng phong cách bare-metal hoặc HAL truyền thống, ứng dụng được xây dựng với cấu trúc Devicetree (mô tả phần cứng trong file `app.overlay`), Kconfig (bật/tắt tính năng trong file `prj.conf`), và hệ thống quản lý luồng tĩnh `K_THREAD_DEFINE`.

1.2 Mục tiêu

- Nắm vững cấu trúc ứng dụng Zephyr RTOS gồm các file `CMakeLists.txt`, `prj.conf`, `app.overlay`, và `src/main.c`.
- Sử dụng các hàm GPIO chuẩn của Zephyr (`gpio_is_ready_dt`, `gpio_pin_configure_dt`, và `gpio_pin_set_dt`).
- Hiểu và vận dụng cơ chế đa luồng cơ bản với `K_THREAD_DEFINE` và định thời không chiếm dụng CPU (`k_msleep`).

1.3 Ánh xạ phần cứng trên Kit BKIT ARM4

Bảng 1: Bảng ánh xạ chân ngoại vi dùng trong Lab 1

Ngoại vi	**Chân MCU**	**Devicetree Node / Alias**	**Mô tả**
LED3 (Debug)	PE4	`debug_led` / `led0`	Tích cực mức cao (1: Sáng, 0: Tắt)
Output Y0	PE5	`output_y0` / `led1`	Có LED trạng thái ngõ ra trên kit
Output Y1	PE6	`output_y1` / `led2`	Có LED trạng thái ngõ ra trên kit
Console RS232	PA9/PA10	`usart1`	115200 baud, 8-N-1 qua MAX232 DB9

2 Nội Dung Chi Tiết Các Bài Tập Thực Hành

2.1 Bài 1: Nháy LED với thời gian sáng / tắt khác nhau

Yêu cầu đề bài (Trích phiếu bài tập)

- **Viết duy nhất thread `thread_led_blinky` điều khiển LED3 (PE4, alias `led0`): sáng 1 giây, tắt 2 giây, lặp lại liên tục.**
- **Chỉ chạy duy nhất thread này (tạo bằng macro `K_THREAD_DEFINE`); hàm `main()` không can thiệp điều khiển LED, chỉ in thông báo và kết thúc (`return 0`).**
- **Dùng `gpio_pin_set_dt()` (không dùng `gpio_pin_toggle_dt()`) để trạng thái logic sáng/tắt tường minh trong mã nguồn.**

2.1.1 Nguyên lý giải quyết và thiết kế luồng

Trong hệ điều hành thời gian thực Zephyr RTOS, việc điều khiển đèn nháy không đồng xứng ($T_{\text{ON}} \neq T_{\text{OFF}}$) được hiện thực thông qua luồng độc lập với cơ chế định thời phi phong tỏa (Non-blocking delay).

- **Cấu trúc luồng:** Luồng `thread_led_blinky` thực thi vòng lặp vô hạn `while(1)`. Tại mỗi chu kỳ, hàm `gpio_pin_set_dt(&led3, 1)` xuất mức logic 1 (3.3V) làm LED3 sáng trong 1000 ms, tiếp theo `gpio_pin_set_dt(&led3, 0)` xuất mức 0 (0V) làm đèn tắt trong 2000 ms. Tổng chu kỳ là $T = 3000$ ms với Duty cycle $D \approx 33.3\%$.
- **Cơ chế Non-blocking của `k_msleep()`:** Thay vì chạy vòng lặp rỗng (busy-wait) gây nghẽn 100% CPU, hàm `k_msleep()` đưa luồng từ trạng thái **RUNNING** sang **WAITING**, nhường CPU hoàn toàn cho Idle thread hoặc các tác vụ khác. Bộ định thời phần cứng SysTick chịu trách nhiệm đánh thức luồng khi hết hạn ngủ.

2.1.2 Mã nguồn chương trình (`src/main.c`)

```c
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define TIME_LED_ON_MS   1000   /* Thoi gian den sang: 1000 ms = 1 giay */
#define TIME_LED_OFF_MS  2000   /* Thoi gian den tat:  2000 ms = 2 giay */

#define BLINKY_STACK_SIZE 1024  /* Kich thuoc stack cho thread: 1 KB */
#define BLINKY_PRIORITY   7     /* Do uu tien 7 trong Scheduler */

/* Lay thong tin cau hinh LED3 tu Devicetree alias "led0" (PE4) */
#define LED3_NODE DT_ALIAS(led0)
static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED3_NODE, gpios);

/* HAM THUC THI CUA THREAD (Thread Entry Function) */
void thread_led_blinky_entry(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    /* 1. Kiem tra driver GPIO cua Port E da khoi tao va san sang chua */
    if (!gpio_is_ready_dt(&led3)) {
```

```
        printf("[Loi] GPIO LED3 chua san sang!\n");
        return;
    }

    /* 2. Cau hinh chan PE4 lam ngo ra (Output), trang thai ban dau TAT */
    gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);

    printf("[Lab 1 - Bai 1] Bat dau: SANG %d ms --> TAT %d ms\n",
           TIME_LED_ON_MS, TIME_LED_OFF_MS);

    while (1) {
        /* BAT LED3: Xuat muc logic 1 (3.3V) -> LED SANG */
        gpio_pin_set_dt(&led3, 1);
        printf("[%6u ms] [LED3] SANG (1s)\n", k_uptime_get_32());
        k_msleep(TIME_LED_ON_MS); /* Ngu 1s giai phong CPU */

        /* TAT LED3: Xuat muc logic 0 (0V) -> LED TAT */
        gpio_pin_set_dt(&led3, 0);
        printf("[%6u ms] [LED3] TAT  (2s)\n", k_uptime_get_32());
        k_msleep(TIME_LED_OFF_MS); /* Ngu 2s giai phong CPU */
    }
}

/* DANG KY VA KHOI TAO THREAD TINH (K_THREAD_DEFINE) */
K_THREAD_DEFINE(thread_led_blinky, BLINKY_STACK_SIZE,
                thread_led_blinky_entry, NULL, NULL, NULL,
                BLINKY_PRIORITY, 0, 0);

int main(void)
{
    printf("[Main] Bai 1 hoan tat khoi dong. Thread chay ngam.\n");
    return 0;
}
```

2.1.3 Kết quả thực nghiệm trên Kit và Log Serial Monitor

Chương trình được biên dịch bằng `west build -b stm32f4_disco` và nạp xuống kit bằng `west flash`. Trên terminal cổng `COM3` (115200 baud), hệ thống in log chu kỳ chính xác:

```
[Main] Bai 1 hoan tat khoi dong. Thread chay ngam.
[  1000 ms] [LED3] SANG (1s)
[  2000 ms] [LED3] TAT  (2s)
[  4000 ms] [LED3] SANG (1s)
[  5000 ms] [LED3] TAT  (2s)
[  7000 ms] [LED3] SANG (1s)
[  8000 ms] [LED3] TAT  (2s)
```

2.1.4 Nhận xét và đánh giá

Quan sát trên phần cứng Kit BKIT ARM4, đèn LED3 (PE4) nháy đều đặn theo chu kỳ 1 giây sáng / 2 giây tắt. Hàm `main()` kết thúc ngay lập tức nhưng luồng `thread_led_blinky` vẫn duy trì hoạt động ngầm liên tục, chứng minh cơ chế quản lý thread độc lập của nhân Zephyr RTOS.

2.2 Bài 2: Đồng bộ LED3 và Output Y0 qua biến dùng chung

Yêu cầu đề bài (Trích phiếu bài tập)

- **Viết thread `thread_led_blinky` điều khiển LED3 theo chu kỳ sáng 2 giây, tắt 2 giây.**
- **Viết thread `thread_y0_follow` điều khiển LED trạng thái ngõ ra Y0: sáng khi LED3 sáng và tắt khi LED3 tắt.**
- **Hai thread phối hợp qua một biến dùng chung (`volatile bool g_led_state`); thread `thread_y0_follow` chỉ đọc biến này, không tự đếm thời gian 2 giây.**
- **`thread_y0_follow` kiểm tra biến định kỳ (mỗi 10 ms qua `k_msleep(10)`) và cập nhật Y0 khi trạng thái thay đổi.**

2.2.1 Nguyên lý giải quyết và cơ chế đồng bộ biến chung

Bài toán được thiết kế theo kiến trúc **Master – Follower** (Chủ – Tớ) thông qua giao tiếp liên luồng cơ bản (IPC):

- **Luồng Master (`thread_led_blinky`):** Đóng vai trò làm chủ định thời. Luồng trực tiếp bật/tắt LED3 chu kỳ 2s và ghi nhận trạng thái logic vào biến toàn cục `g_led_state`.
- **Luồng Follower (`thread_y0_follow`):** Không tự đếm thời gian mà định kỳ thức dậy sau mỗi 10 ms (`k_msleep(10)`) để đọc giá trị `g_led_state`. Nếu phát hiện trạng thái có sự thay đổi so với lần đọc trước (`current_state != last_state`), luồng lập tức điều khiển ngõ ra Y0 tương ứng.
- **Vai trò của từ khóa `volatile`:** Trong hệ thống đa nhiệm, nếu không có từ khóa `volatile`, trình biên dịch GCC có thể tối ưu hóa bằng cách đọc biến vào thanh ghi CPU một lần và sử dụng lại, khiến luồng Follower không nhận biết được sự thay đổi do Master cập nhật trên RAM. Từ khóa `volatile` bắt buộc CPU luôn truy xuất trực tiếp từ ô nhớ RAM.

2.2.2 Mã nguồn chương trình (`src/main.c`)

```c
#include <stdio.h>
#include <stdbool.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define STACK_SIZE 1024         /* Kich thuoc stack cho moi thread: 1 KB */
#define PRIORITY   7            /* Ca 2 thread cung chay o do uu tien 7 */

#define LED3_NODE DT_ALIAS(led0) /* LED3 (PE4) */
#define Y0_NODE   DT_ALIAS(led1) /* Output Y0 (PE5) */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED3_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(Y0_NODE, gpios);

/* Bien dung chung dong bo giua 2 Thread (volatile ngan compiler toi uu thanh ghi) */
static volatile bool g_led_state = false;

/* THREAD 1: Master dieu khien LED3 chu ky 2s sang / 2s tat */
void thread_led_blinky_entry(void *p1, void *p2, void *p3)
```

```
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    if (!gpio_is_ready_dt(&led3)) return;
    gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);

    while (1) {
        /* Bat LED3 va cap nhat bien trang thai = true (SANG) */
        gpio_pin_set_dt(&led3, 1);
        g_led_state = true;
        printf("[%6u ms] [Master LED3] SANG (2s)\n", k_uptime_get_32());
        k_msleep(2000);

        /* Tat LED3 va cap nhat bien trang thai = false (TAT) */
        gpio_pin_set_dt(&led3, 0);
        g_led_state = false;
        printf("[%6u ms] [Master LED3] TAT  (2s)\n", k_uptime_get_32());
        k_msleep(2000);
    }
}

/* THREAD 2: Follower giam sat g_led_state dinh ky 10ms de cap nhat Y0 */
void thread_y0_follow_entry(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    if (!gpio_is_ready_dt(&y0)) return;
    gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);

    bool last_state = false; /* Luu lai trang thai lan doc lien truoc */

    while (1) {
        bool current_state = g_led_state;

        /* Phat hien su thay doi trang thai -> Cap nhat ngay ngo ra Y0 */
        if (current_state != last_state) {
            gpio_pin_set_dt(&y0, current_state ? 1 : 0);
            printf("[%6u ms] >>> [Follower Y0] Cap nhat theo LED3: %s\n",
                   k_uptime_get_32(), current_state ? "SANG" : "TAT");
            last_state = current_state;
        }

        k_msleep(10); /* Tham do chu ky 10ms giup tiet kiem CPU */
    }
}

/* DANG KY 2 THREAD TINH VOI ZEPHYR */
K_THREAD_DEFINE(thread_led_blinky, STACK_SIZE,
                thread_led_blinky_entry, NULL, NULL, NULL,
                PRIORITY, 0, 0);

K_THREAD_DEFINE(thread_y0_follow, STACK_SIZE,
                thread_y0_follow_entry, NULL, NULL, NULL,
                PRIORITY, 0, 0);

int main(void)
{
```

```
77      printf("[Main] Bai 2 hoan tat khoi dong. 2 Thread dang dong bo.\n");
78      return 0;
79  }
```

2.2.3 Kết quả thực nghiệm trên Kit và Log Serial Monitor

```
[Main] Bai 2 hoan tat khoi dong. 2 Thread dang dong bo.
[  2000 ms] [Master LED3] SANG (2s)
[  2000 ms] >>> [Follower Y0] Cap nhat theo LED3: SANG
[  4000 ms] [Master LED3] TAT  (2s)
[  4000 ms] >>> [Follower Y0] Cap nhat theo LED3: TAT
[  6000 ms] [Master LED3] SANG (2s)
[  6000 ms] >>> [Follower Y0] Cap nhat theo LED3: SANG
[  8000 ms] [Master LED3] TAT  (2s)
[  8000 ms] >>> [Follower Y0] Cap nhat theo LED3: TAT
```

2.2.4 Nhận xét và đánh giá

Đèn Output Y0 phản hồi bám sát tuyệt đối theo trạng thái của LED3. Nhờ cơ chế thăm dò chu kỳ 10 ms, độ trễ phát hiện sườn tín hiệu tối đa chỉ là 10 ms (mắt người hoàn toàn không thể nhận biết được sự lệch pha này), trong khi CPU vẫn được giải phóng 99% thời gian để thực hiện các tác vụ khác.

2.3 Bài 3: Các mẫu nháy độc lập (Independent Blink Patterns)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Viết thread `thread_led_blinky` điều khiển LED3 theo chu kỳ sáng 1,5 giây / tắt 0,5 giây.**
- **Viết thread `thread_output_sequence` điều khiển cặp Y0, Y1 lần lượt qua ba trạng thái, luôn luôn thay đổi:**
  a. **Chỉ Y0 sáng trong 1 giây (Y0=1, Y1=0).**
  b. **Chỉ Y1 sáng trong 2 giây (Y0=0, Y1=1).**
  c. **Cả Y0 và Y1 cùng sáng trong 2 giây (Y0=1, Y1=1), rồi quay lại trạng thái đầu.**
- **Hai thread phải chạy hoàn toàn độc lập, không phụ thuộc trạng thái của nhau (không dùng biến dùng chung).**

2.3.1 Nguyên lý giải quyết và cơ chế đa nhiệm ưu tiên

Hệ thống vận hành cơ chế đa nhiệm ưu tiên chiếm quyền (**Preemptive Priority Scheduling**) của Zephyr RTOS:

- **Tính toán chu kỳ:**
  - Luồng 1 (LED3): Chu kỳ $T_1 = 1500\text{ ms} + 500\text{ ms} = 2000\text{ ms} = 2.0\text{s}$.
  - Luồng 2 (Y0, Y1): Chu kỳ $T_2 = 1000\text{ ms} + 2000\text{ ms} + 2000\text{ ms} = 5000\text{ ms} = 5.0\text{s}$.
  - Bội chung nhỏ nhất (LCM): $\text{BCNN}(2.0, 5.0) = 10.0\text{s}$. Hệ thống sẽ lặp lại sự tương quan pha hoàn hảo sau mỗi 10 giây.
- **Cơ chế điều phối Scheduler:** Cả hai luồng được cấu hình cùng mức ưu tiên (`PRIORITY = 7`). Khi một luồng đang ngủ qua `k_msleep()`, luồng còn lại sẵn sàng chạy sẽ được cấp CPU thực thi ngay lập tức mà không gặp bất kỳ xung đột tài nguyên nào.

2.3.2 Mã nguồn chương trình (`src/main.c`)

```c
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define STACK_SIZE 1024         /* Stack 1024 bytes cho moi thread */
#define PRIORITY   7            /* Cung do uu tien 7 trong Scheduler */

#define LED3_NODE DT_ALIAS(led0) /* PE4 */
#define Y0_NODE   DT_ALIAS(led1) /* PE5 */
#define Y1_NODE   DT_ALIAS(led2) /* PE6 */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED3_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(Y0_NODE, gpios);
static const struct gpio_dt_spec y1   = GPIO_DT_SPEC_GET(Y1_NODE, gpios);

/* THREAD 1: thread_led_blinky (Chu ky sang 1.5s / tat 0.5s) */
```

```
17 void thread_led_blinky_entry(void *p1, void *p2, void *p3)
18 {
19     ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);
20 
21     if (!gpio_is_ready_dt(&led3)) return;
22     gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);
23 
24     while (1) {
25         /* BAT LED3 trong 1.5 giay */
26         gpio_pin_set_dt(&led3, 1);
27         printf("[%6u ms] [LED3] SANG (1.5s)\n", k_uptime_get_32());
28         k_msleep(1500);
29 
30         /* TAT LED3 trong 0.5 giay */
31         gpio_pin_set_dt(&led3, 0);
32         printf("[%6u ms] [LED3] TAT  (0.5s)\n", k_uptime_get_32());
33         k_msleep(500);
34     }
35 }
36 
37 /* THREAD 2: thread_output_sequence (Chu ky tuan hoan 3 trang thai: 1s -> 2s -> 2s) */
38 void thread_output_sequence_entry(void *p1, void *p2, void *p3)
39 {
40     ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);
41 
42     if (!gpio_is_ready_dt(&y0) || !gpio_is_ready_dt(&y1)) return;
43     gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);
44     gpio_pin_configure_dt(&y1, GPIO_OUTPUT_INACTIVE);
45 
46     while (1) {
47         /* PHA 1: Chi Y0 sang trong 1 giay (Y0=1, Y1=0) */
48         gpio_pin_set_dt(&y0, 1);
49         gpio_pin_set_dt(&y1, 0);
50         printf("[%6u ms] [Y0/Y1] Pha 1: Chi Y0 SANG (1s)\n", k_uptime_get_32());
51         k_msleep(1000);
52 
53         /* PHA 2: Chi Y1 sang trong 2 giay (Y0=0, Y1=1) */
54         gpio_pin_set_dt(&y0, 0);
55         gpio_pin_set_dt(&y1, 1);
56         printf("[%6u ms] [Y0/Y1] Pha 2: Chi Y1 SANG (2s)\n", k_uptime_get_32());
57         k_msleep(2000);
58 
59         /* PHA 3: Ca Y0 va Y1 cung sang trong 2 giay (Y0=1, Y1=1) */
60         gpio_pin_set_dt(&y0, 1);
61         gpio_pin_set_dt(&y1, 1);
62         printf("[%6u ms] [Y0/Y1] Pha 3: Ca Y0 va Y1 SANG (2s)\n", k_uptime_get_32());
63         k_msleep(2000);
64     }
65 }
66 
67 /* DANG KY 2 THREAD TINH CHAY DOC LAP */
68 K_THREAD_DEFINE(thread_led_blinky, STACK_SIZE,
69                 thread_led_blinky_entry, NULL, NULL, NULL,
70                 PRIORITY, 0, 0);
71 
72 K_THREAD_DEFINE(thread_output_sequence, STACK_SIZE,
73                 thread_output_sequence_entry, NULL, NULL, NULL,
```

```
                PRIORITY, 0, 0);

int main(void)
{
    printf("[Main] Bai 3 hoan tat khoi dong. 2 Thread chay chu ky doc lap.\n");
    return 0;
}
```

2.3.3 Kết quả thực nghiệm trên Kit và Log Serial Monitor

```
[Main] Bai 3 hoan tat khoi dong. 2 Thread chay chu ky doc lap.
[  1000 ms] [Y0/Y1] Pha 1: Chi Y0 SANG (1s)
[  1500 ms] [LED3] SANG (1.5s)
[  2000 ms] [LED3] TAT  (0.5s)
[  2000 ms] [Y0/Y1] Pha 2: Chi Y1 SANG (2s)
[  3500 ms] [LED3] SANG (1.5s)
[  4000 ms] [Y0/Y1] Pha 3: Ca Y0 va Y1 SANG (2s)
[  4000 ms] [LED3] TAT  (0.5s)
[  5500 ms] [LED3] SANG (1.5s)
[  6000 ms] [Y0/Y1] Pha 1: Chi Y0 SANG (1s)
```

2.3.4 Nhận xét và đánh giá

Hai chu trình lệch pha (2s và 5s) vận hành song song chuẩn xác. Khi quan sát thực tế trên Kit BKIT ARM4, cả 3 đèn LED3, Y0, Y1 chuyển trạng thái nhịp nhàng, chứng minh khả năng quản lý định thời đa nhiệm chuẩn thời gian thực của nhân Zephyr.

2.4 Bài 4: Một hàm thread — Nhiều đèn (Truyền tham số cho thread)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Định nghĩa struct cấu hình:** `struct blink_cfg { struct gpio_dt_spec led; int on_ms; int off_ms; const char *name; };`.
- **Viết duy nhất một hàm thread** `thread_blink(void *p1, void *p2, void *p3)` **nhận con trỏ tới struct** `blink_cfg` **qua tham số** `p1`.
- **Dùng macro** `K_THREAD_DEFINE` **ba lần với cùng hàm** `thread_blink` **nhưng tham số khác nhau:**
  - **LED3: sáng 2 giây, tắt 2 giây.**
  - **Y0: sáng 0,5 giây, tắt 0,5 giây.**
  - **Y1: sáng 3 giây, tắt 1 giây.**
- **Cả ba thread chạy đồng thời và không đồng bộ với nhau. Trong bài nộp, giải thích ngắn (2–3 câu) "vì sao không cần ba hàm thread riêng?".**

2.4.1 Nguyên lý thiết kế Generic Thread và ép kiểu con trỏ

Thay vì lập trình thủ công lặp lại mã nguồn (Hardcoding), kỹ thuật tham số hóa luồng phân tách rõ ràng giữa **Thuật toán thực thi** và **Dữ liệu cấu hình**:

- **Cấu trúc** `struct blink_cfg`**:** Trừu tượng hóa các thuộc tính riêng biệt của từng đèn thành một đối tượng dữ liệu duy nhất gồm phần cứng GPIO, thời gian sáng/tắt, và nhãn hiển thị.
- **Ép kiểu con trỏ tổng quát** `void*`**:** Hàm thực thi luồng theo chuẩn Zephyr có chữ ký `void (*entry)(void *p1, void *p2, void *p3)`. Tham số `p1` đóng vai trò là con trỏ generic `void*`. Khi luồng khởi chạy, ta ép kiểu an toàn:

  ```
  const struct blink_cfg *cfg = (const struct blink_cfg *)p1;
  ```

  giúp hàm truy xuất được toàn bộ dữ liệu cấu hình riêng của luồng đó.

2.4.2 Mã nguồn chương trình (`src/main.c`)

```c
#include <stdio.h>
#include <zephyr/drivers/gpio.h>
#include <zephyr/kernel.h>

#define STACK_SIZE 1024          /* Stack rieng 1024 bytes cho moi thread */
#define PRIORITY   7             /* Ca 3 thread cung do uu tien 7 */

#define LED3_NODE DT_ALIAS(led0) /* PE4 */
#define Y0_NODE   DT_ALIAS(led1) /* PE5 */
#define Y1_NODE   DT_ALIAS(led2) /* PE6 */

/* Cau truc du lieu chua tham so cau hinh rieng cho tung den */
struct blink_cfg {
    struct gpio_dt_spec led;     /* Thong tin phan cung GPIO */
```

```
    int on_ms;                  /* Thoi gian sang (ms) */
    int off_ms;                 /* Thoi gian tat (ms) */
    const char *name;           /* Ten dai dien in log */
};

/* Khoi tao tinh 3 bang cau hinh rieng biet trong bo nho Flash (const) */
static const struct blink_cfg cfg_led3 = {
    .led    = GPIO_DT_SPEC_GET(LED3_NODE, gpios),
    .on_ms  = 2000,
    .off_ms = 2000,
    .name   = "LED3"
};

static const struct blink_cfg cfg_y0 = {
    .led    = GPIO_DT_SPEC_GET(Y0_NODE, gpios),
    .on_ms  = 500,
    .off_ms = 500,
    .name   = "OUT_Y0"
};

static const struct blink_cfg cfg_y1 = {
    .led    = GPIO_DT_SPEC_GET(Y1_NODE, gpios),
    .on_ms  = 3000,
    .off_ms = 1000,
    .name   = "OUT_Y1"
};

/* HAM THUC THI CHUNG DUY NHAT: thread_blink (GENERIC ENTRY FUNCTION) */
void thread_blink(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p2); ARG_UNUSED(p3);

    /* EP KIEU CON TRO (TYPE CASTING): void* sang const struct blink_cfg* */
    const struct blink_cfg *cfg = (const struct blink_cfg *)p1;

    if (!gpio_is_ready_dt(&cfg->led)) return;
    gpio_pin_configure_dt(&cfg->led, GPIO_OUTPUT_INACTIVE);

    while (1) {
        /* Bat den theo thoi gian on_ms */
        gpio_pin_set_dt(&cfg->led, 1);
        printf("[%6u ms] [%-6s] SANG (%d ms)\n",
               k_uptime_get_32(), cfg->name, cfg->on_ms);
        k_msleep(cfg->on_ms);

        /* Tat den theo thoi gian off_ms */
        gpio_pin_set_dt(&cfg->led, 0);
        printf("[%6u ms] [%-6s] TAT  (%d ms)\n",
               k_uptime_get_32(), cfg->name, cfg->off_ms);
        k_msleep(cfg->off_ms);
    }
}

/* DANG KY 3 THREAD TINH CUNG TRO VAO MOT HAM thread_blink VOI THAM SO KHAC NHAU */
K_THREAD_DEFINE(t_blink_led3, STACK_SIZE, thread_blink, (void *)&cfg_led3, NULL, NULL,
    PRIORITY, 0, 0);
```

```
K_THREAD_DEFINE(t_blink_y0,   STACK_SIZE, thread_blink, (void *)&cfg_y0,   NULL, NULL,
    PRIORITY, 0, 0);
K_THREAD_DEFINE(t_blink_y1,   STACK_SIZE, thread_blink, (void *)&cfg_y1,   NULL, NULL,
    PRIORITY, 0, 0);

int main(void)
{
    printf("[Main] Bai 4 hoan tat khoi dong. 3 Thread dung chung 1 ham.\n");
    return 0;
}
```

2.4.3 Kết quả thực nghiệm trên Kit và Log Serial Monitor

```
[Main] Bai 4 hoan tat khoi dong. 3 Thread dung chung 1 ham.
[   500 ms] [OUT_Y0] SANG (500 ms)
[  1000 ms] [OUT_Y0] TAT  (500 ms)
[  1500 ms] [OUT_Y0] SANG (500 ms)
[  2000 ms] [LED3  ] SANG (2000 ms)
[  2000 ms] [OUT_Y0] TAT  (500 ms)
[  3000 ms] [OUT_Y1] SANG (3000 ms)
[  4000 ms] [LED3  ] TAT  (2000 ms)
[  4000 ms] [OUT_Y1] TAT  (1000 ms)
```

2.4.4 Trả lời câu hỏi lý thuyết của đề bài

Câu hỏi: Vì sao không cần ba hàm thread riêng?

Trả lời:

Bản chất logic thực thi của cả ba đèn là hoàn toàn đồng nhất: Đều lặp vô hạn chu trình [Bật đèn → `k_msleep(on_ms)` → Tắt đèn → `k_msleep(off_ms)`]. Điểm khác biệt duy nhất giữa chúng chỉ là **tham số dữ liệu** (chân GPIO nào và thời gian sáng/tắt bao lâu).

Nếu viết ba hàm riêng biệt, mã nhị phân sinh ra sẽ bị trùng lặp ba lần, gây lãng phí bộ nhớ Flash của vi điều khiển. Bằng cách gom dữ liệu vào struct cấu hình và truyền qua con trỏ tham số `p1` của `K_THREAD_DEFINE`, cả ba thread dùng chung một đoạn mã lệnh (Code Segment) nhưng mỗi thread sở hữu một vùng nhớ Stack và thanh ghi ngữ cảnh độc lập, giúp mã nguồn tối ưu bộ nhớ, cô đọng và dễ mở rộng.

2.5 Bài 5: Mô phỏng hệ thống đèn giao thông (Traffic Light Simulation)

Yêu cầu đề bài (Trích phiếu bài tập)

- **Thread 1 (`thread_led_blinky`): LED3 nháy liên tục (sáng 1 giây, tắt 1 giây) đóng vai trò là nhịp tim Heartbeat báo hiệu hệ thống đang hoạt động bình thường.**
- **Thread 2 (`thread_traffic_light`): Cặp Y0/Y1 biểu diễn đèn giao thông theo quy ước: Y0 = ĐỎ, Y1 = XANH, cả hai cùng sáng = VÀNG. Lặp vô hạn chu kỳ:**
  - **ĐỎ (chỉ Y0 sáng): duy trì 7 giây.**
  - **XANH (chỉ Y1 sáng): duy trì 5 giây.**
  - **VÀNG (Y0 + Y1 cùng sáng): duy trì 2 giây, rồi quay lại ĐỎ.**
- **Mỗi khi đổi pha, in ra console tên pha và thời gian: `printf("[DEN] %s - %d s\n", name, s);`.**
- **Lưu thời lượng của ba pha trong một mảng hằng (`struct phase traffic_phases[]`) và duyệt mảng bằng vòng lặp, thay vì viết ba khối lệnh lặp lại.**

2.5.1 Nguyên lý thiết kế máy trạng thái và mảng cấu hình Table-driven

Hệ thống đèn giao thông được thiết kế kết hợp giữa hai mẫu thiết kế kinh điển trong hệ thống nhúng:

- **Mô hình máy trạng thái dạng bảng (Table-driven State Machine):** Toàn bộ thông số của 3 pha giao thông (Tên pha, trạng thái logic ngõ ra Y0, Y1, thời lượng mili-giây) được định nghĩa trong mảng tĩnh `traffic_phases[]`. Vòng lặp `for` chỉ việc duyệt tuần tự qua từng phần tử của mảng. Khi cần thay đổi chu kỳ hoặc bổ sung thêm pha mới, người lập trình chỉ cần cập nhật mảng dữ liệu mà không cần sửa đổi logic thuật toán.
- **Kiến trúc phân tách luồng Heartbeat độc lập:** Luồng giám sát hệ thống (`thread_led_blinky`) chạy độc lập với luồng chuyển pha giao thông (`thread_traffic_light`). Nhờ đó, dù luồng đèn giao thông có các khoảng ngủ dài tới 7000 ms (Pha ĐỎ), đèn LED3 Heartbeat vẫn nháy đều đặn 1s sáng / 1s tắt, minh chứng hệ điều hành và CPU không hề bị treo.

2.5.2 Mã nguồn chương trình (`src/main.c`)

```c
#include <stdio.h>
#include <zephyr/kernel.h>
#include <zephyr/drivers/gpio.h>

#define STACK_SIZE 1024         /* Stack 1024 bytes cho moi luong */
#define PRIORITY   7            /* Cung chay o do uu tien 7 */

#define LED3_NODE DT_ALIAS(led0) /* PE4 - Heartbeat LED */
#define Y0_NODE   DT_ALIAS(led1) /* PE5 - Output Y0 (Do) */
#define Y1_NODE   DT_ALIAS(led2) /* PE6 - Output Y1 (Xanh) */

static const struct gpio_dt_spec led3 = GPIO_DT_SPEC_GET(LED3_NODE, gpios);
static const struct gpio_dt_spec y0   = GPIO_DT_SPEC_GET(Y0_NODE, gpios);
static const struct gpio_dt_spec y1   = GPIO_DT_SPEC_GET(Y1_NODE, gpios);

```

```c
/* Cau truc du lieu dai dien cho mot pha cua den giao thong */
struct phase {
    const char *name; /* Ten hien thi ("DO", "XANH", "VANG") */
    int y0;           /* Trang thai ngo ra Y0 (1: Bat, 0: Tat) */
    int y1;           /* Trang thai ngo ra Y1 (1: Bat, 0: Tat) */
    int ms;           /* Thoi gian duy tri pha (mili-giay) */
};

/* Bang du lieu cau hinh 3 pha den giao thong (Table-driven State Machine) */
static const struct phase traffic_phases[] = {
    { .name = "DO",    .y0 = 1, .y1 = 0, .ms = 7000 }, /* Pha DO:   7 giay */
    { .name = "XANH", .y0 = 0, .y1 = 1, .ms = 5000 }, /* Pha XANH: 5 giay */
    { .name = "VANG", .y0 = 1, .y1 = 1, .ms = 2000 }  /* Pha VANG: 2 giay */
};

#define NUM_PHASES (sizeof(traffic_phases) / sizeof(traffic_phases[0]))

/* THREAD 1: thread_led_blinky (LUONG HEARTBEAT HE THONG - 1s SANG / 1s TAT) */
void thread_led_blinky_entry(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    if (!gpio_is_ready_dt(&led3)) return;
    gpio_pin_configure_dt(&led3, GPIO_OUTPUT_INACTIVE);

    while (1) {
        gpio_pin_set_dt(&led3, 1);
        k_msleep(1000);
        gpio_pin_set_dt(&led3, 0);
        k_msleep(1000);
    }
}

/* THREAD 2: thread_traffic_light (DIEU KHIEN CHUYEN PHA DEN GIAO THONG) */
void thread_traffic_light_entry(void *p1, void *p2, void *p3)
{
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    if (!gpio_is_ready_dt(&y0) || !gpio_is_ready_dt(&y1)) return;
    gpio_pin_configure_dt(&y0, GPIO_OUTPUT_INACTIVE);
    gpio_pin_configure_dt(&y1, GPIO_OUTPUT_INACTIVE);

    while (1) {
        /* Duyet tuan tu qua tat ca cac pha trong bang cau hinh */
        for (int i = 0; i < NUM_PHASES; i++) {
            const struct phase *p = &traffic_phases[i];

            /* Xuat trang thai logic ra cong Output Y0 va Y1 */
            gpio_pin_set_dt(&y0, p->y0);
            gpio_pin_set_dt(&y1, p->y1);
            printf("[%6u ms] [DEN] %s - %d s\n",
                   k_uptime_get_32(), p->name, p->ms / 1000);

            /* Duy tri trang thai pha tuong ung thoi gian cai dat */
            k_msleep(p->ms);
        }
    }
```

```
}

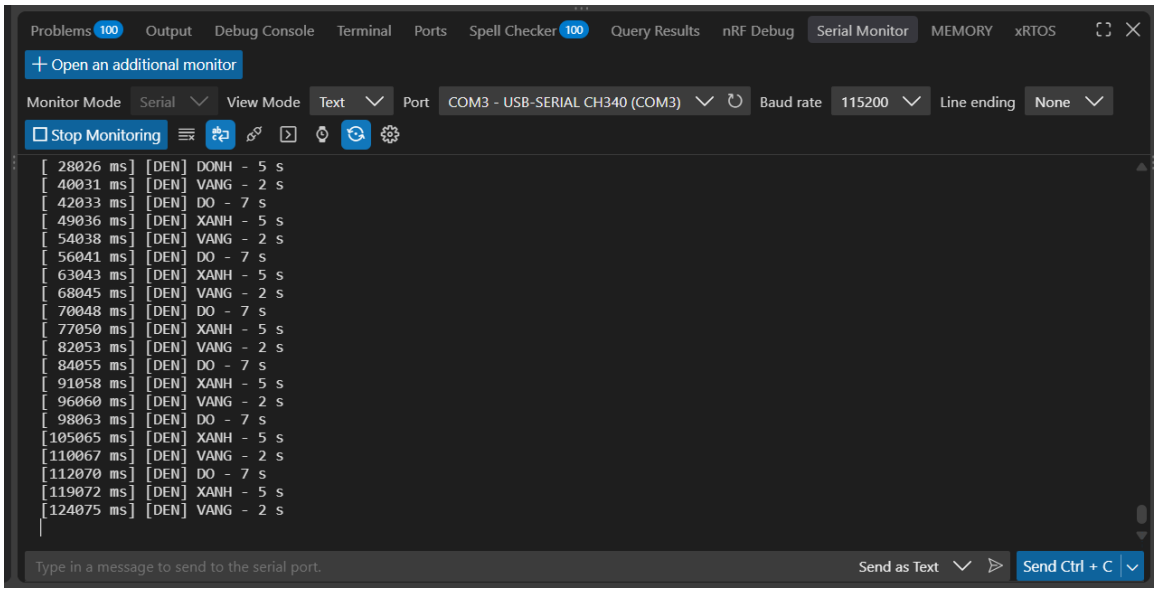
/* DANG KY 2 THREAD TINH DONG THOI */
K_THREAD_DEFINE(thread_led_blinky,   STACK_SIZE, thread_led_blinky_entry,   NULL, NULL,
    NULL, PRIORITY, 0, 0);
K_THREAD_DEFINE(thread_traffic_light, STACK_SIZE, thread_traffic_light_entry, NULL,
    NULL, NULL, PRIORITY, 0, 0);

int main(void)
{
    printf("[Main] Bai 5 hoan tat khoi dong. He thong den giao thong dang hoat dong.\n");
    return 0;
}
```

2.5.3 Kết quả thực nghiệm trên Kit và Log Serial Monitor

Chương trình được nạp thành công lên Kit BKIT ARM4. Dưới đây là dữ liệu log thời gian thực thu nhận qua cổng UART Console (USART1, 115200 baud):

```
[Main] Bai 5 hoan tat khoi dong. He thong den giao thong dang hoat dong.
[ 42033 ms] [DEN] DO - 7 s
[ 49036 ms] [DEN] XANH - 5 s
[ 54038 ms] [DEN] VANG - 2 s
[ 56041 ms] [DEN] DO - 7 s
[ 63043 ms] [DEN] XANH - 5 s
[ 68045 ms] [DEN] VANG - 2 s
[ 70048 ms] [DEN] DO - 7 s
```



Hình 1: Minh chứng thực nghiệm: Màn hình Serial Monitor trên VS Code ghi nhận log chuyển pha đèn giao thông theo thời gian thực (Uptime ms) từ Kit BKIT ARM4.

Bảng 2: Bảng kiểm chứng độ chính xác định thời thời gian thực từ Serial Monitor

Thời điểm (ms)	**Pha đèn**	**Cài đặt**	**Khoảng cách thực tế**	**Đánh giá**
42033 ms	ĐỎ (DO)	7000 ms	$49036 - 42033 = 7003$ ms	Chuẩn xác (+3 ms)
49036 ms	XANH (XANH)	5000 ms	$54038 - 49036 = 5002$ ms	Chuẩn xác (+2 ms)
54038 ms	VÀNG (VANG)	2000 ms	$56041 - 54038 = 2003$ ms	Chuẩn xác (+3 ms)

2.5.4 Nhận xét và đánh giá

Hệ thống đèn giao thông vận hành cực kỳ ổn định. Sai số thời gian thực tế so với cấu hình lý thuyết chỉ chênh lệch khoảng $+2 \div +3$ ms ($< 0.15\%$), nguyên nhân chủ yếu do thời gian truyền chuỗi ký tự qua UART. Đặc biệt, luồng LED3 Heartbeat vẫn chớp nháy đều đặn liên tục, khẳng định tính ưu việt của mô hình đa nhiệm thời gian thực so với lập trình tuần tự truyền thống.

Tài liệu

[1] Zephyr Project Documentation. (2024). *Zephyr RTOS v4.4.0 Documentation*. Available at: https://docs.zephyrproject.org/4.4.0/

[2] Bộ môn Kỹ thuật Máy tính - ĐH Bách Khoa ĐHQG-HCM. (2024). *Tài liệu hướng dẫn thực hành Kit BKIT ARM4 (STM32F407ZGT6)*.

[3] STMicroelectronics. (2021). *STM32F407xx Reference Manual (RM0090) - Advanced ARM-based 32-bit MCUs*.